



Competitive Programming: Non-Linear DSA #1

Presenter: Dr. Ha Viet Uyen Synh.

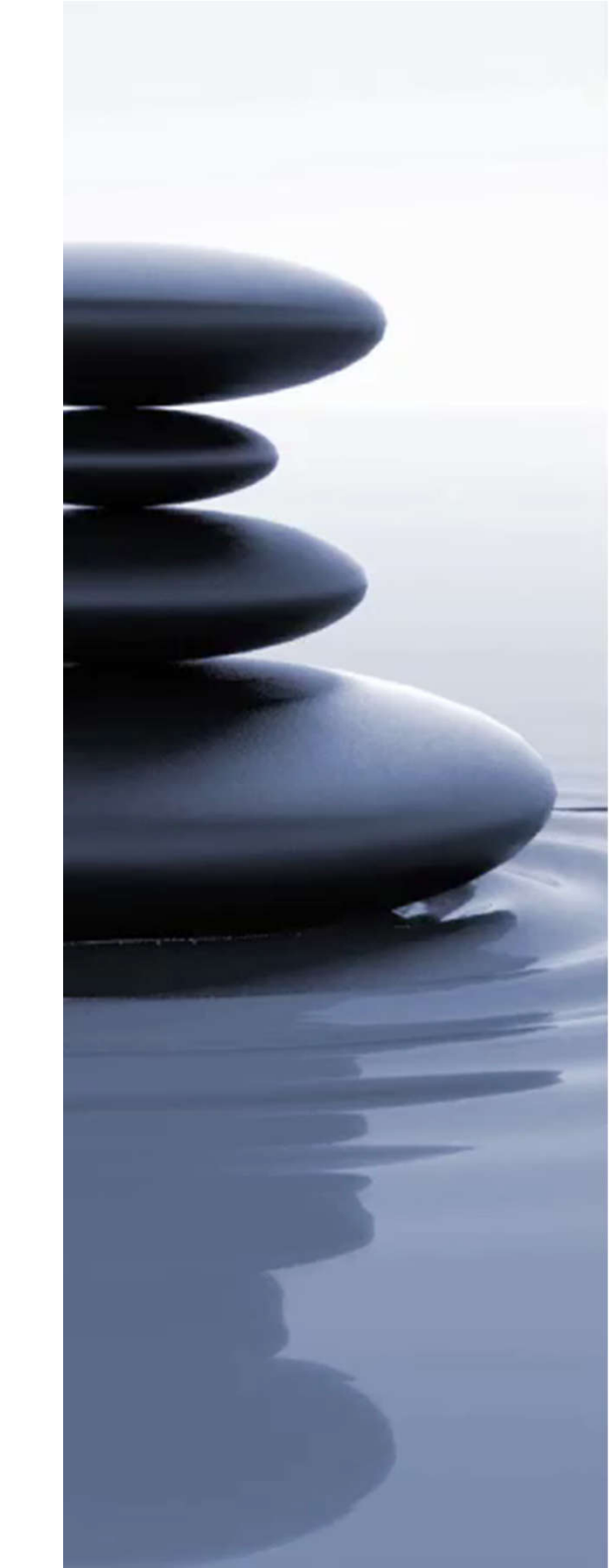


Graph traversal



PART A

BREADTH-FIRST SEARCH



Problem

Given a source vertex s , find the shortest paths (in terms of number of edges) to every reachable vertex from s .

Solution

The procedure we will use will find all vertices reachable at a distance k before discovering reachable vertices at a distance $k+1$. Ultimately, the algorithm will produce a breadth-first tree with s as the root.

During the execution of the algorithm, vertices will be colored (denoted by $u.color$). The colors represent the vertex's current state as follows

white - the vertex is undiscovered (i.e. currently no path has been found to the vertex)

gray - the vertex has been discovered and is on the frontier, i.e. there may be further vertices that can be discovered

black - the vertex has been discovered and has been completely searched

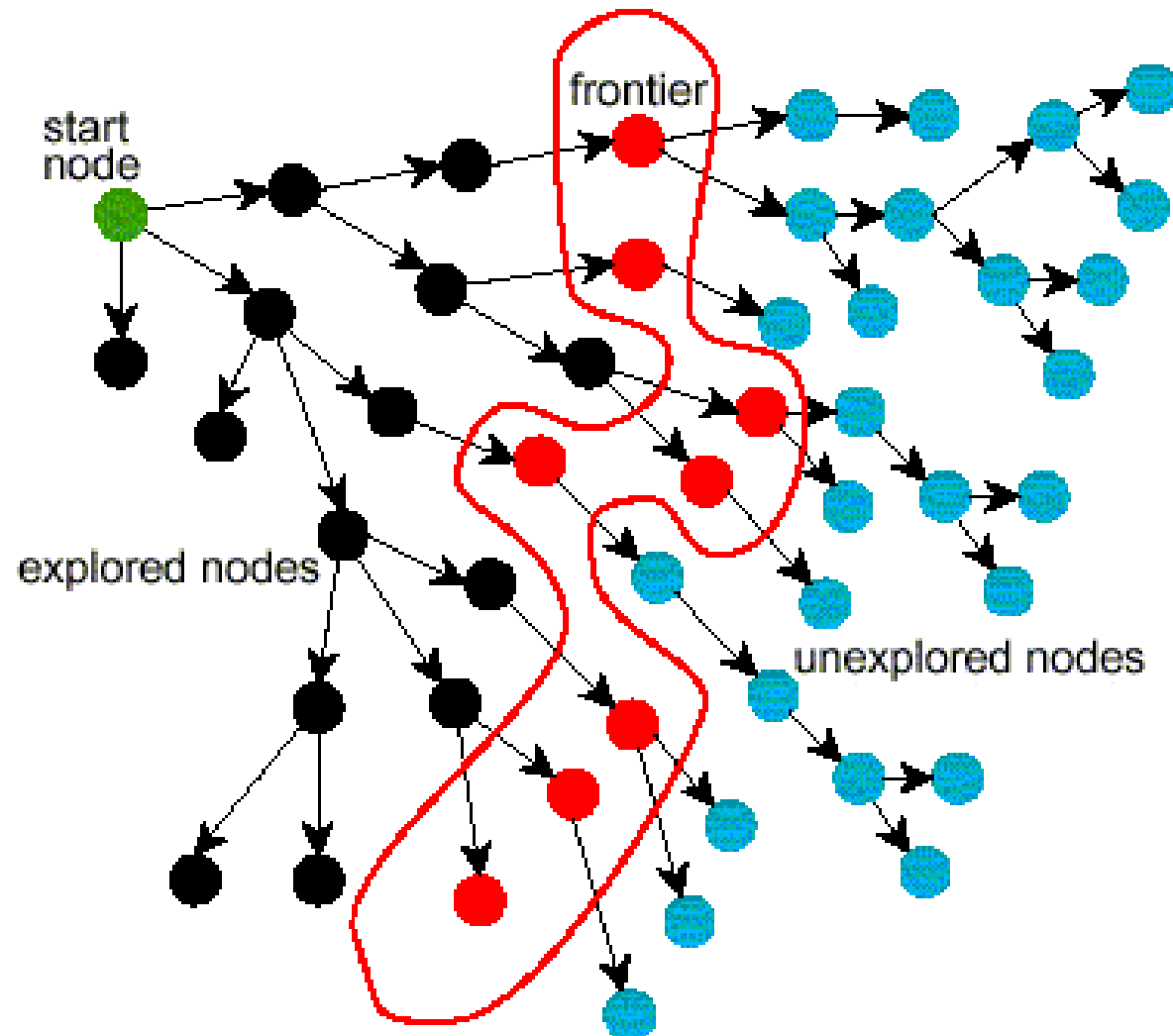
The algorithm also uses two additional fields for each vertex

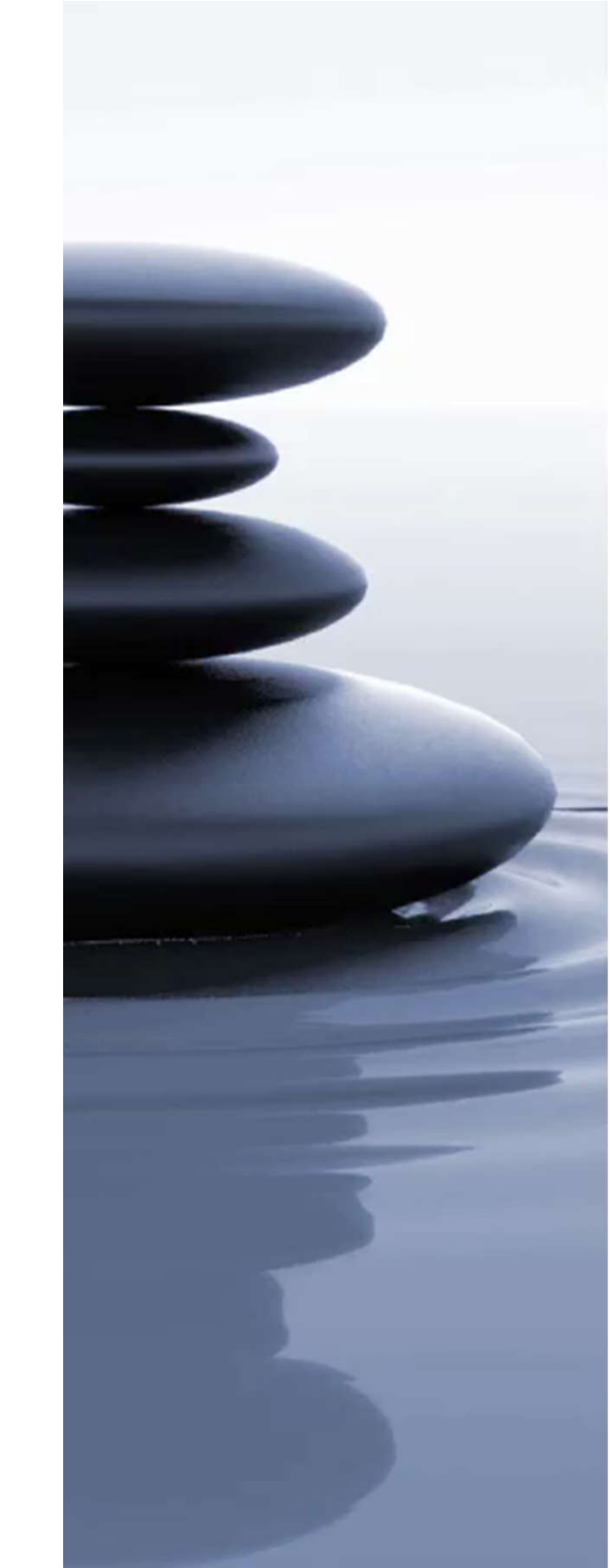
$u.\pi$ - predecessor vertex

$u.d$ - distance when the vertex is first discovered (and is subsequently the shortest distance from the source)

We will employ a **queue Q** which will track which vertices are currently under discovery. Thus vertices that have not yet been placed in Q will be **white**, those that are in Q will be **gray**, and those that have been removed from Q will be **black**.

Problem Solving by Graph Searching

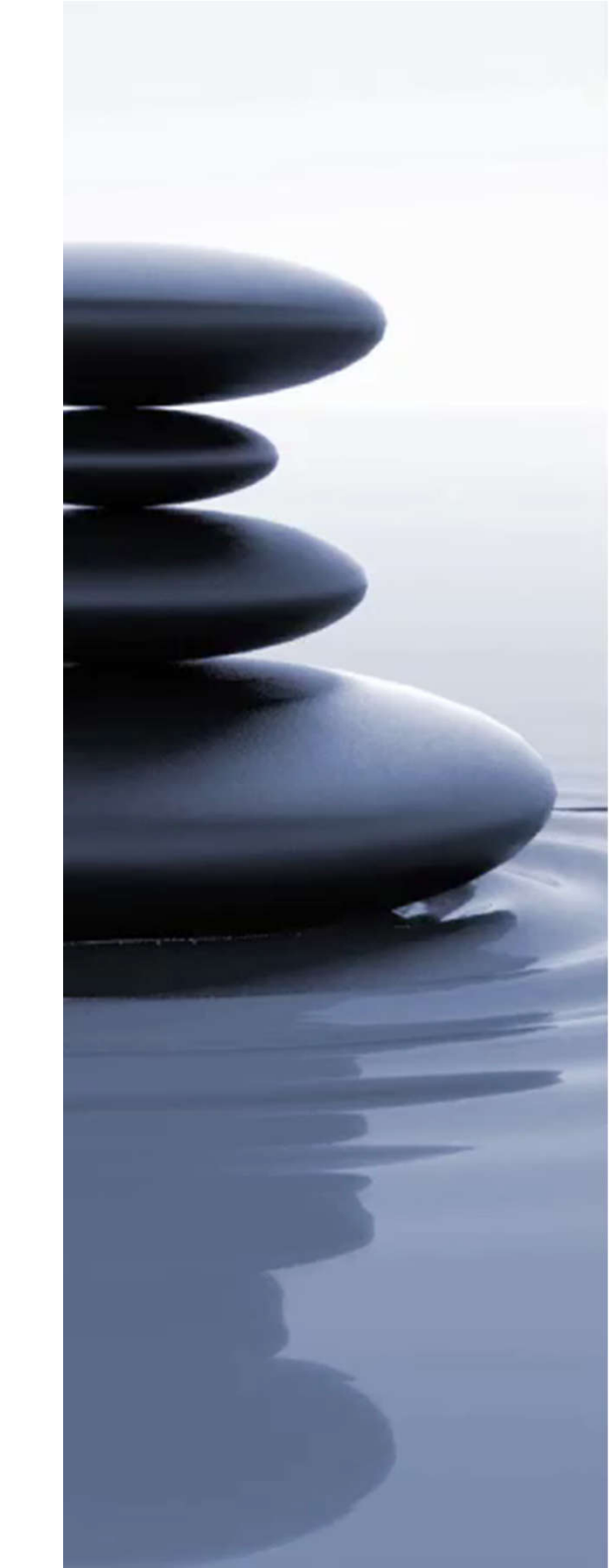




Algorithm

Basically the algorithm performs the following operations:

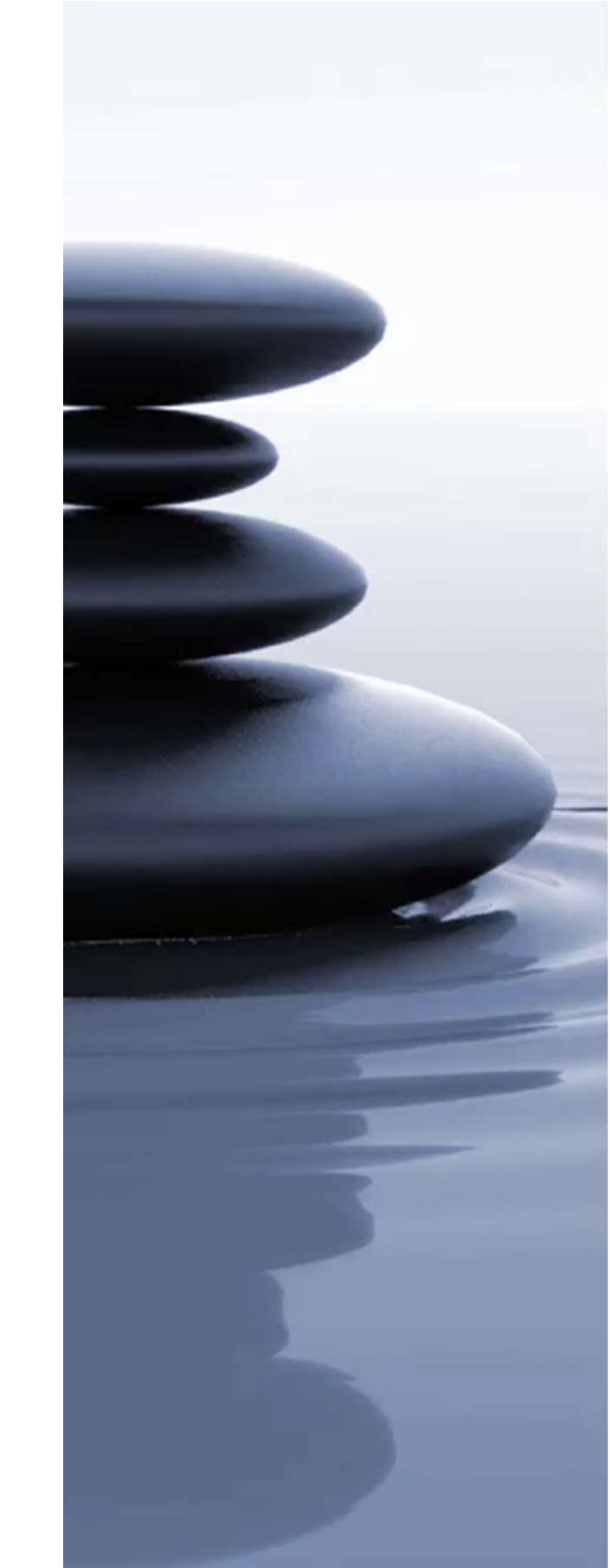
1. Initialize Q with the source vertex s
2. Dequeue the head vertex u from Q and mark as black
3. Queue all white vertices adjacent to u marking them as gray, set their distance to u 's distance + 1, and set their π to u
4. Repeat 2-3 until $Q = \emptyset$



Algorithm

BFS(G, s)

1. for each vertex $u \in G.V - \{s\}$
2. $u.color == WHITE$
3. $u.d = INF$
4. $u.pi = NIL$
5. $s.color = GRAY$
6. $s.d = 0$
7. $s.pi = NIL$
8. $Q = \emptyset$
9. ENQUEUE(Q, s)
10. while $Q \neq \emptyset$
11. $u = DEQUEUE(Q)$
12. for each $v \in G.Adj[u]$
13. if $v.color == WHITE$
14. $v.color = GRAY$
15. $v.d = u.d + 1$
16. $v.pi = u$
17. ENQUEUE(Q, v)
18. $u.color = BLACK$



Implementation

```
vector<vector<int>> adj; // adjacency list representation
int n; // number of nodes
int s; // source vertex
```

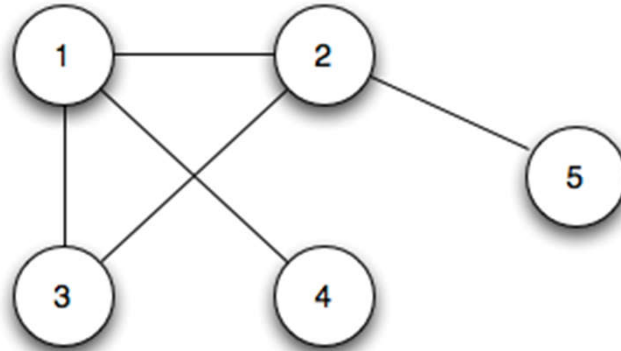
```
queue<int> q;
vector<bool> used(n);
vector<int> d(n), p(n);
```

```
q.push(s);
used[s] = true;
p[s] = -1;
while (!q.empty()) {
    int v = q.front();
    q.pop();
    for (int u : adj[v]) {
        if (!used[u]) {
            used[u] = true;
            q.push(u);
            d[u] = d[v] + 1;
            p[u] = v;
        }
    }
}
```



```
if (!used[u]) {  
    cout << "No path!";  
} else {  
    vector<int> path;  
    for (int v = u; v != -1; v = p[v])  
        path.push_back(v);  
    reverse(path.begin(), path.end());  
    cout << "Path: ";  
    for (int v : path)  
        cout << v << " ";  
}
```

Example



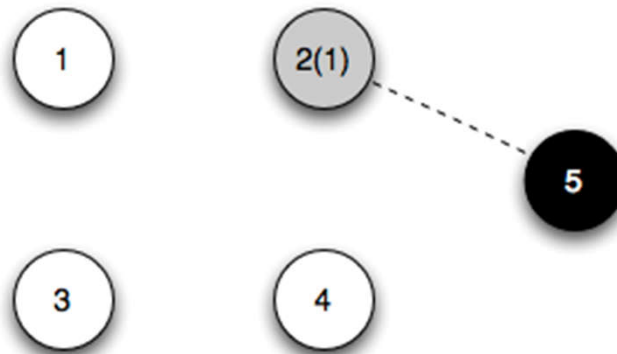
If we select vertex 5 as the source, then $d[5]=0$, $\pi[5]=/$, $Q=\{5\}$, thus the initialization gives



$Q = \{5\}$

	1	2	3	4	5
d	∞	∞	∞	∞	0
π	/	/	/	/	/

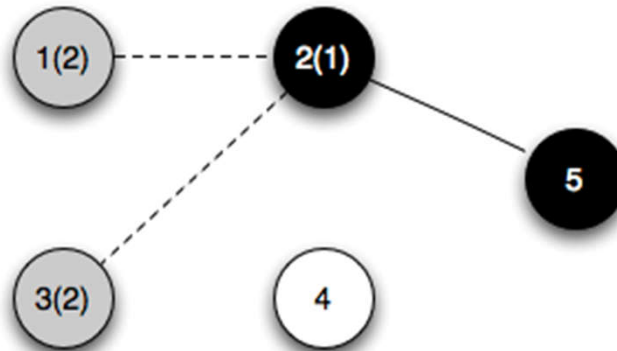
Iteration 1: dequeue vertex 5 and enqueue vertex 2



$Q = \{2\}$

	1	2	3	4	5
d	∞	1	∞	∞	0
π	/	5	/	/	/

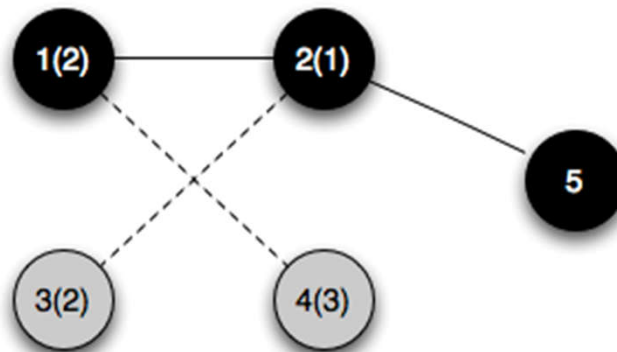
Iteration 2: dequeue vertex 2 and enqueue vertices 1,3



$Q = \{1,3\}$

	1	2	3	4	5
d	2	1	2	∞	0
π	2	5	2	/	/

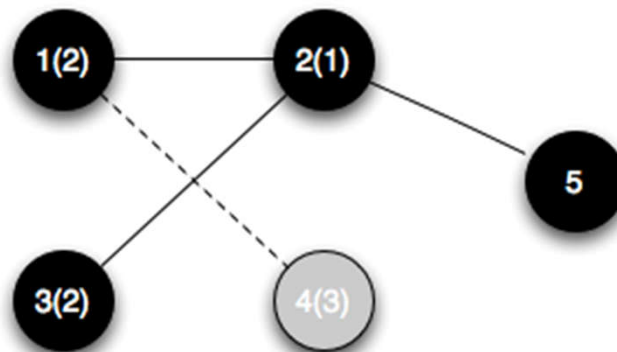
Iteration 3: dequeue vertex 1 and enqueue vertex 4



$Q = \{3,4\}$

	1	2	3	4	5
d	2	1	2	3	0
π	2	5	2	1	/

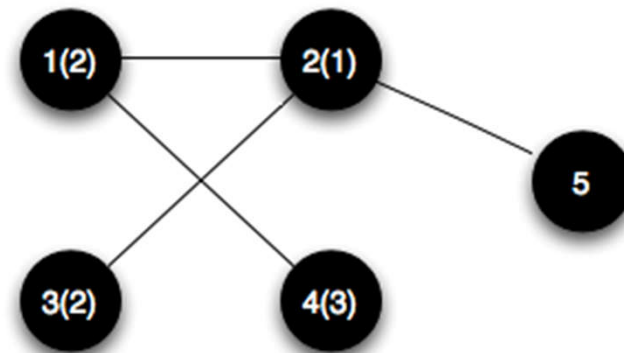
Iteration 4: dequeue vertex 3 and enqueue no vertices



$Q = \{4\}$

	1	2	3	4	5
d	2	1	2	3	0
π	2	5	2	1	/

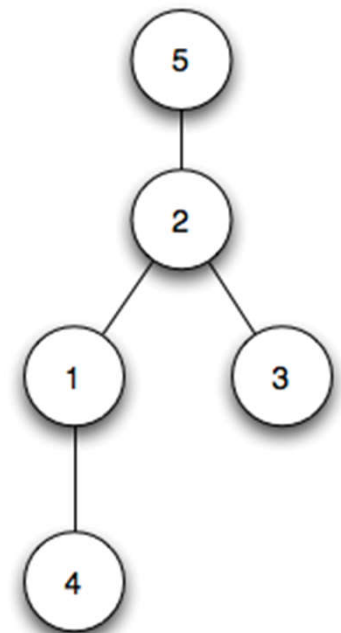
Iteration 5: dequeue vertex 4 and enqueue no vertices (thus leaving the queue empty)



$Q = \emptyset$

	1	2	3	4	5
d	2	1	2	3	0
π	2	5	2	1	/

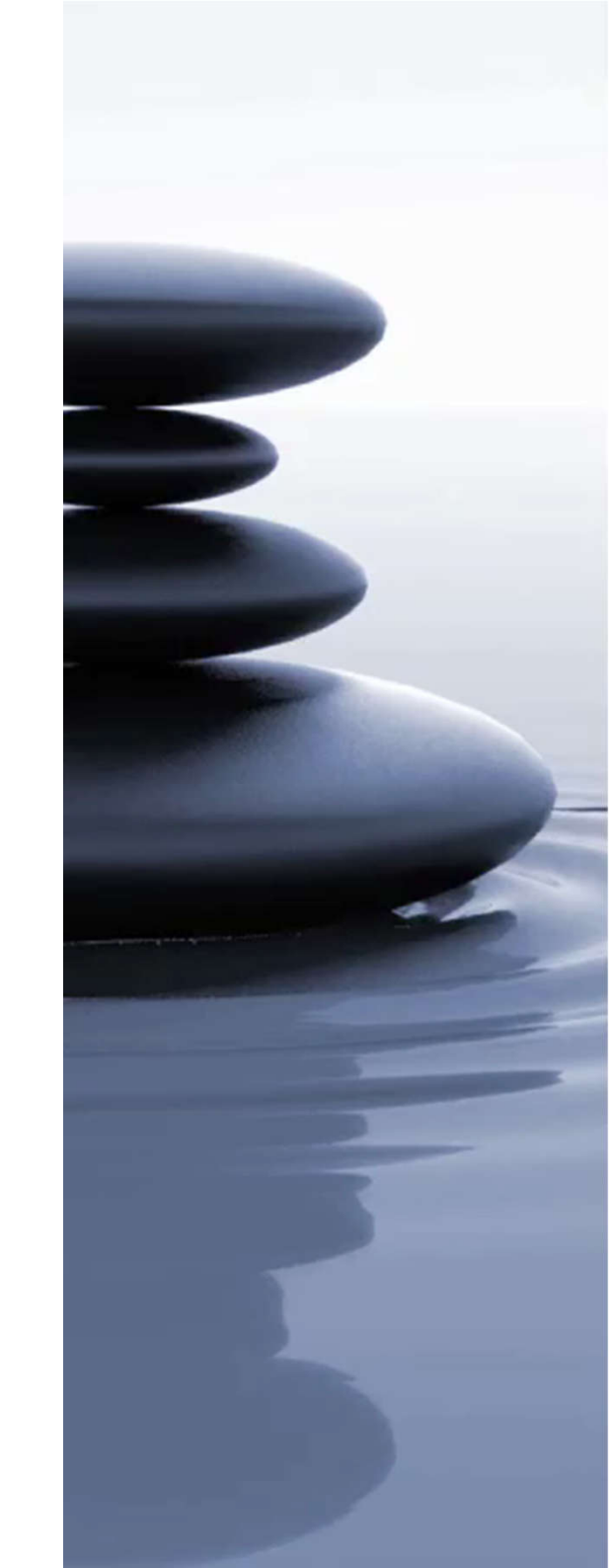
The final predecessor graph, i.e. breadth-first tree, for this BFS is



Implementation

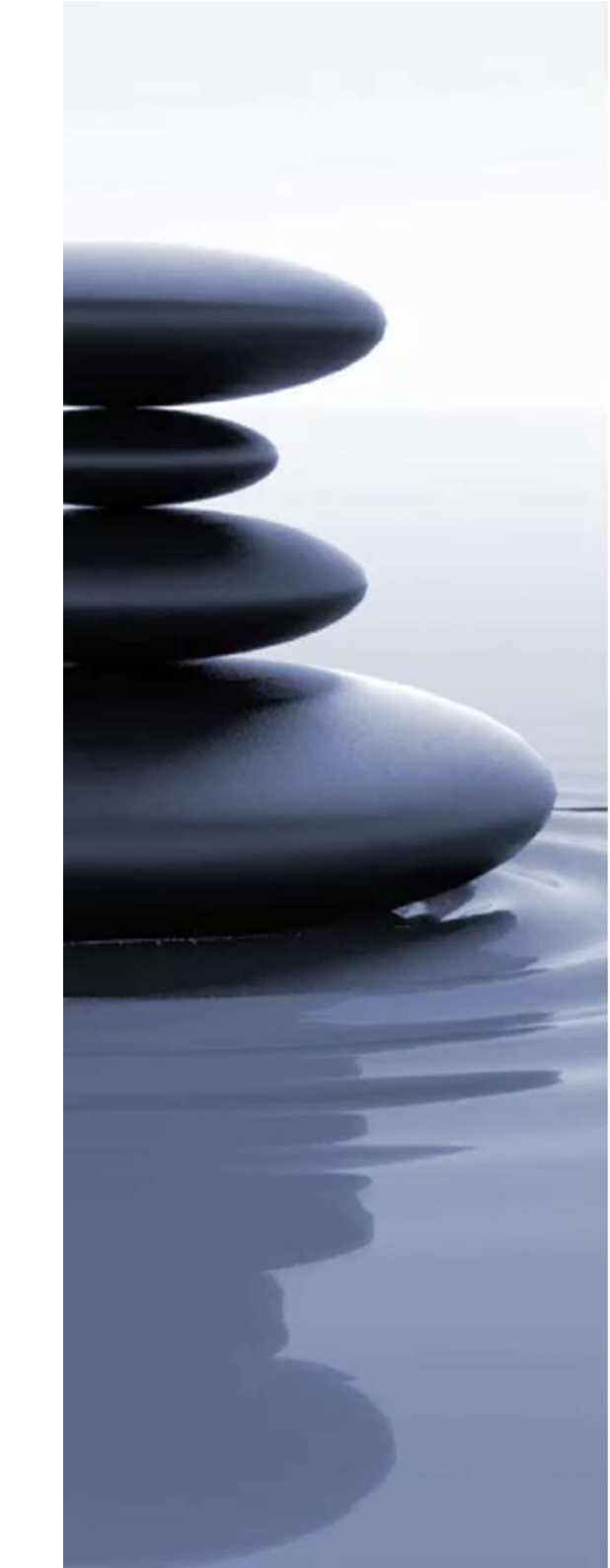
```
// inside int main()---no recursion
vi d(V, INF); d[s] = 0;           // distance from source s to s is 0
queue<int> q; q.push(s);          // start from source

while (!q.empty()) {
    int u = q.front(); q.pop();    // queue: layer by layer!
    for (int j = 0; j < (int)AdjList[u].size(); j++) {
        ii v = AdjList[u][j];    // for each neighbor of u
        if (d[v.first] == INF) { // if v.first is unvisited + reachable
            d[v.first] = d[u] + 1; // make d[v.first] != INF to flag it
            q.push(v.first);       // enqueue v.first for the next iteration
        }
    }
}
```



Applications

- Find the shortest path from a source to other vertices in an unweighted graph.
- Find all connected components in an undirected graph in $O(n+m)$ time: To do this, we just run BFS starting from each vertex, except for vertices which have already been visited from previous runs. Thus, we perform normal BFS from each of the vertices, but do not reset the array used[] each and every time we get a new connected component, and the total running time will still be $O(n+m)$ (performing multiple BFS on the graph without zeroing the array used[] is called a series of breadth first searches).
- Finding a solution to a problem or a game with the least number of moves, if each state of the game can be represented by a vertex of the graph, and the transitions from one state to the other are the edges of the graph.
- Finding the shortest path in a graph with weights 0 or 1: This requires just a little modification to normal breadth-first search: Instead of maintaining array used[], we will now check if the distance to vertex is shorter than current found distance, then if the current edge is of zero weight, we add it to the front of the queue else we add it to the back of the queue. This modification is explained in more detail in the article [0-1 BFS](#).
- Finding the shortest cycle in a directed unweighted graph: Start a breadth-first search from each vertex. As soon as we try to go from the current vertex back to the source vertex, we have found the shortest cycle containing the source vertex. At this point we can stop the BFS, and start a new BFS from the next vertex. From all such cycles (at most one from each BFS) choose the shortest.
- Find all the edges that lie on any shortest path between a given pair of vertices (a,b). To do this, run two breadth first searches: one from a and one from b. Let da[] be the array containing shortest distances obtained from the first BFS (from a) and db[] be the array containing shortest distances obtained from the second BFS from b. Now for every edge (u,v) it is easy to check whether that edge lies on any shortest path between a and b: the criterion is the condition $da[u]+1+db[v]=da[b]$.
- Find all the vertices on any shortest path between a given pair of vertices (a,b). To accomplish that, run two breadth first searches: one from a and one from b. Let da[] be the array containing shortest distances obtained from the first BFS (from a) and db[] be the array containing shortest distances obtained from the second BFS (from b). Now for each vertex it is easy to check whether it lies on any shortest path between a and b: the criterion is the condition $da[v]+db[v]=da[b]$.
- Find the shortest path of even length from a source vertex s to a target vertex t in an unweighted graph: For this, we must construct an auxiliary graph, whose vertices are the state (v,c), where v - the current node, c=0 or c=1 - the current parity. Any edge (a,b) of the original graph in this new column will turn into two edges ((u,0),(v,1)) and ((u,1),(v,0)). After that we run a BFS to find the shortest path from the starting vertex (s,0) to the end vertex (t,0).



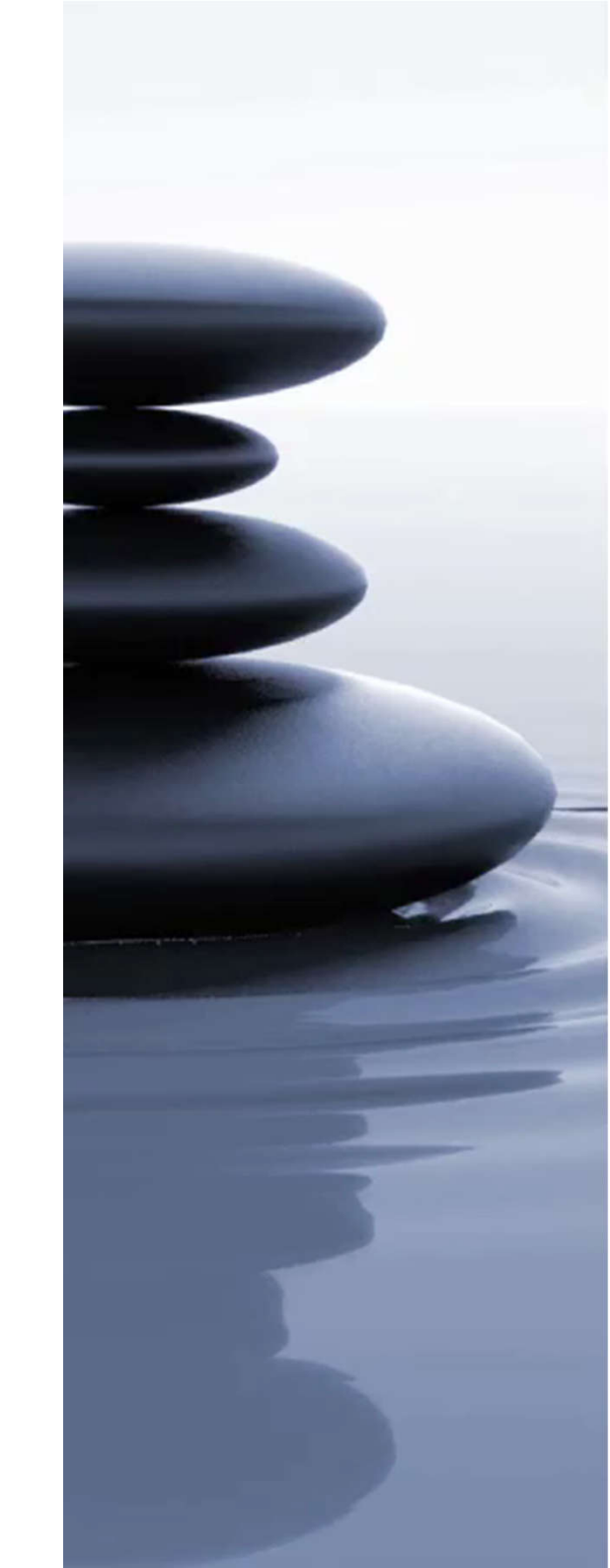
Problems

- [SPOJ: AKBAR](#)
- [SPOJ: NAKANJ](#)
- [SPOJ: WATER](#)
- [SPOJ: MICE AND MAZE](#)
- [Timus: Caravans](#)
- [DevSkill - Halloween Party \(archived\)](#)
- [DevSkill - Ohani And The Link Cut Tree \(archived\)](#)
- [SPOJ - Spiky Mazes](#)
- [SPOJ - Four Chips \(hard\)](#)
- [SPOJ - Inversion Sort](#)
- [Codeforces - Shortest Path](#)
- [SPOJ - Yet Another Multiple Problem](#)
- [UVA 11392 - Binary 3xType Multiple](#)
- [UVA 10968 - KuPellaKeS](#)
- [Codeforces - Police Stations](#)
- [Codeforces - Okabe and City](#)
- [SPOJ - Find the Treasure](#)
- [Codeforces - Bear and Forgotten Tree 2](#)
- [Codeforces - Cycle in Maze](#)
- [UVA - 11312 - Flipping Frustration](#)
- [SPOJ - Ada and Cycle](#)
- [CSES - Labyrinth](#)
- [CSES - Message Route](#)
- [CSES - Monsters](#)



PART B

DEPTH-FIRST SEARCH



Problem

Given a source vertex s , follow a path to maximal depth and then backtrack to follow any remaining paths available from proceeding vertices. If after exploring the source any vertices remain undiscovered, begin another depth first search at an undiscovered vertex repeating until all vertices in the graph are fully explored.

Solution

The procedure we will use is similar to BFS, but instead of finding all vertices at distance k before locating vertices at distance $k+1$ we will simply follow any path until no new vertices can be found along that path. Thus the resulting depth-first trees can vary depending on the order in which the vertices are traversed.

During the execution of the algorithm, vertices will again be colored (denoted by $u.color$) similarly to BFS. As a review, the colors represent the vertex's current state as follows

white - the vertex is undiscovered (i.e. currently no path has been found to the vertex)

gray - the vertex has been discovered and is on the frontier, i.e. there may be further vertices that can be discovered

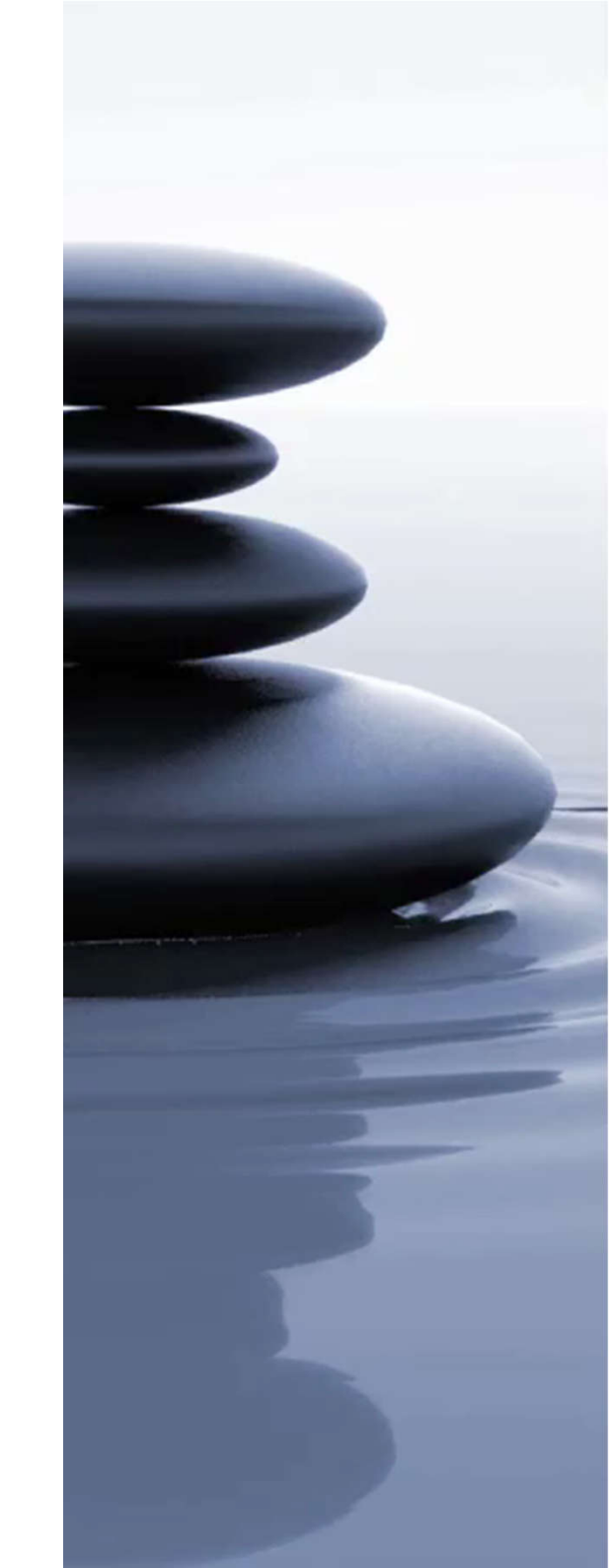
black - the vertex has been discovered and has been completely searched

The DFS algorithm uses similar fields as BFS but also requires a third one f as follows

$u.\pi$ - predecessor vertex

$u.d$ - timestamp when the vertex is first discovered (and is subsequently colored gray)

$u.f$ - timestamp when the vertex has been fully explored (and is subsequently colored black) $\Rightarrow u.d < u.f$



At each step in the algorithm we increment the timestamp giving
 $1 \leq u.d < u.f \leq 2|V|$ for all $u \in V$

Therefore a vertex will be white before $u.d$, gray between $u.d$ and $u.f$, and black after $u.f$.

After DFS, all of the edges in the graph can then be classified as either:

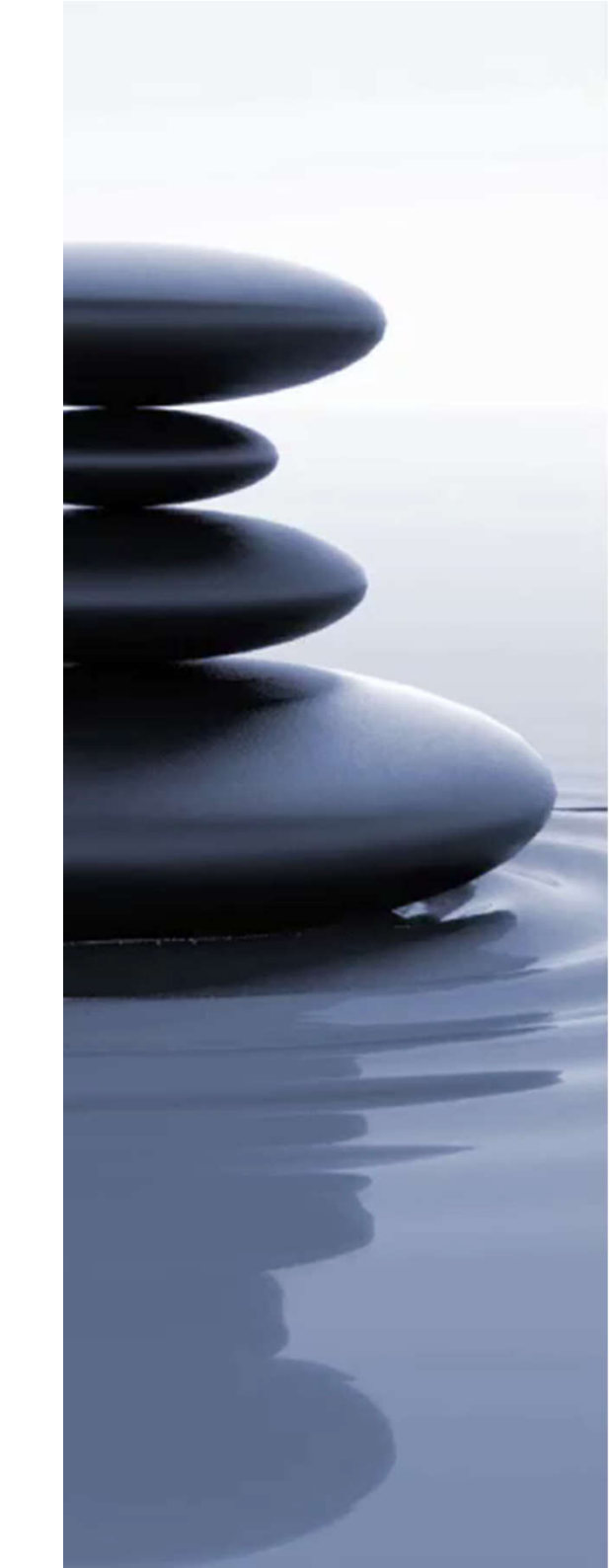
Tree edge - edge in one of the depth-first trees, i.e. v is first discovered from $u \Rightarrow$ if v is white when (u,v) is explored, then (u,v) is a tree edge

Back edge - non-tree edges connecting u to an ancestor of $v \Rightarrow$ if v is gray when (u,v) is explored, then (u,v) is a back edge

Forward edge - non-tree edges connecting u to a descendant of $v \Rightarrow$ if v is black when (u,v) is explored and $u.d < v.d$, then (u,v) is a forward edge

Cross edge - all other edges $\Rightarrow$ if v is black when (u,v) is explored and $v.d < u.d$, then (u,v) is a cross edge

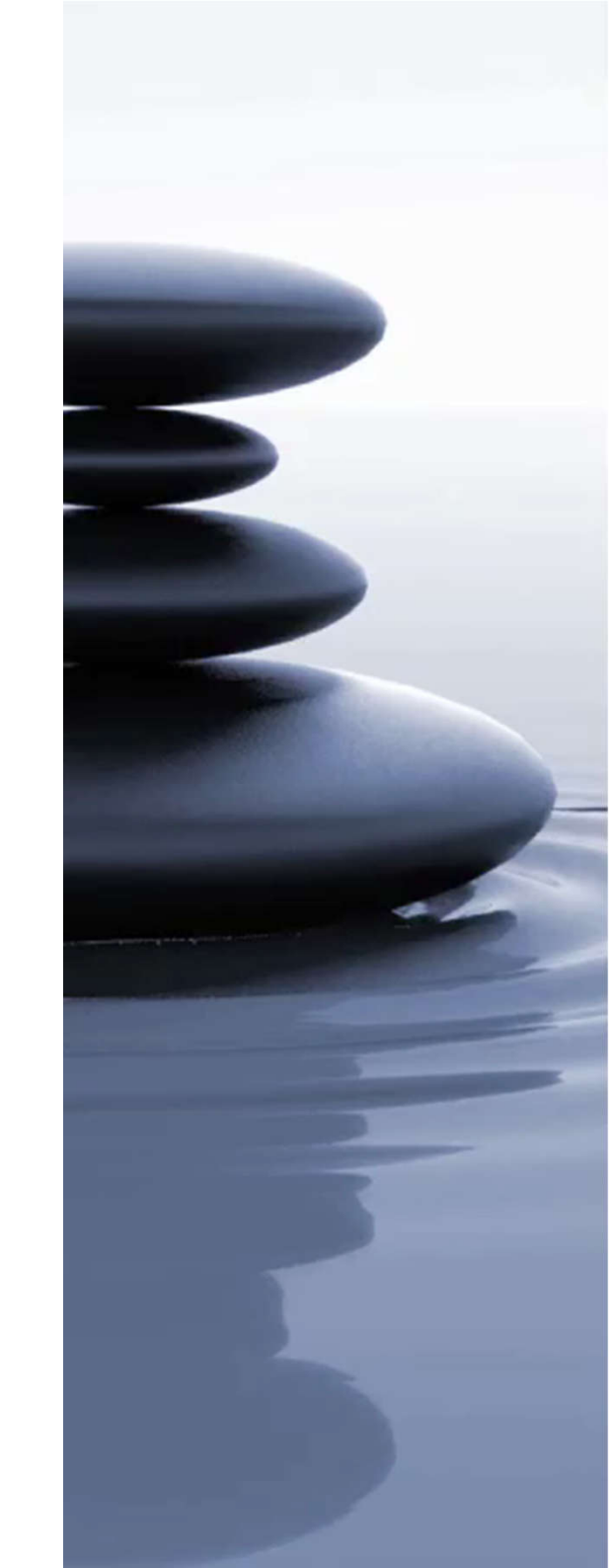
Note that forward and cross edges cannot occur if the graph is undirected.



Algorithm

Basically the algorithm performs the following operations:

1. Initialize all the vertices to white
2. Recurse on any white vertex u coloring subsequent discovered vertices gray and incrementing the timestamp
3. When no further recursion is possible (no other vertices in the current adjacency list are white) return up a level and recurse on any other white vertices
4. Repeat 2-3 until no white vertices remain in the graph



Algorithm

DFS(G)

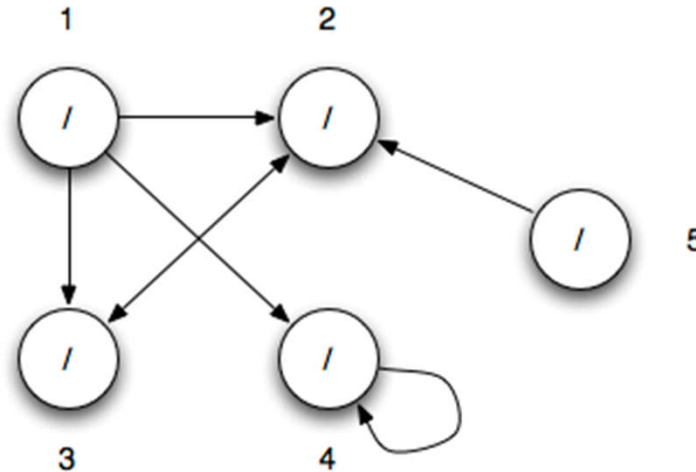
1. for each vertex $u \in G.V$
2. $u.color = WHITE$
3. $u.pi = NIL$
4. $time = 0$
5. for each vertex $u \in G.V$
6. if $u.color == WHITE$
7. DFS-VISIT(G,u)

DFS-VISIT(G,u)

1. $time = time + 1$
2. $u.d = time$
3. $u.color = GRAY$
4. for each $v \in G.Adj[u]$
5. if $v.color == WHITE$
6. $v.pi = u$
7. DFS-VISIT(G,v)
8. $u.color = BLACK$
9. $time = time + 1$
10. $u.f = time$

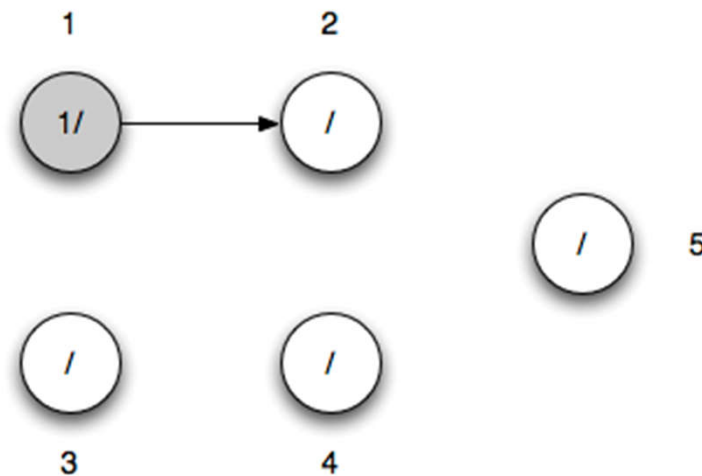
Example

Consider the five node (directed) graph



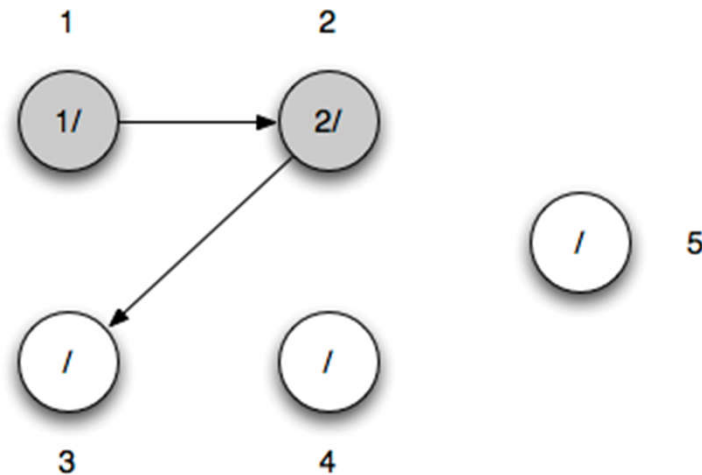
	1	2	3	4	5
π	/	/	/	/	/

If we start at vertex 1 (and follow vertices in increasing numerical order), then
Timestep 1: $u_1.d = 1$ and we follow edge (u_1, u_2)



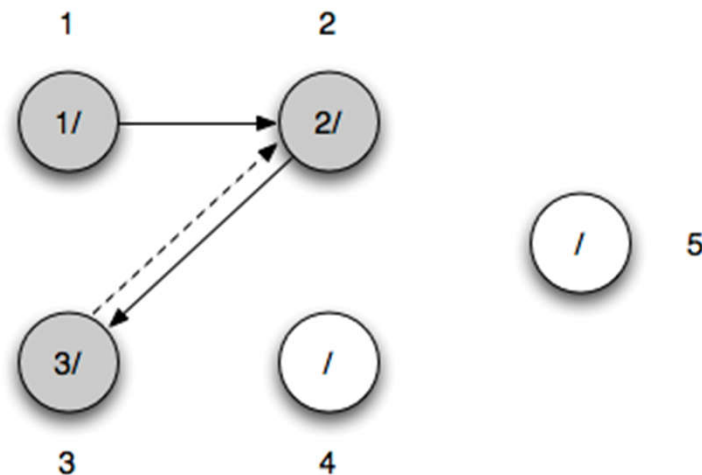
	1	2	3	4	5
d	1	/	/	/	/
f	/	/	/	/	/
π	/	/	/	/	/

Timestep 2: $u_2.d = 2$ and we follow edge (u_2, u_3)



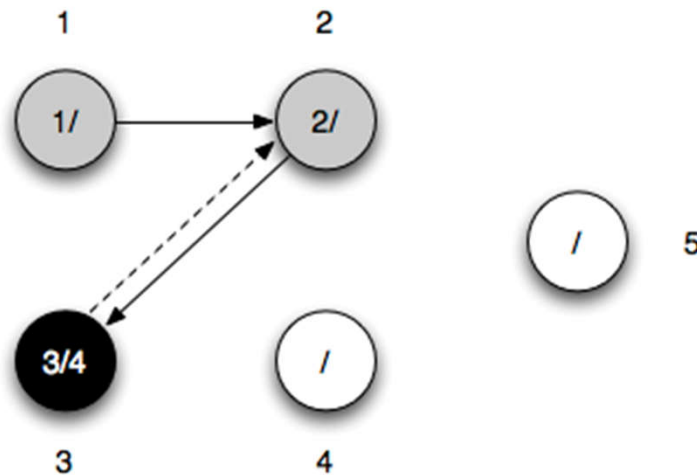
	1	2	3	4	5
d	/	1	2	/	/
f	/	/	/	/	/
π	/	1	2	/	/

Timestep 3: $u_3.d = 3$ (note edge (u_3, u_2) is a *back edge*)



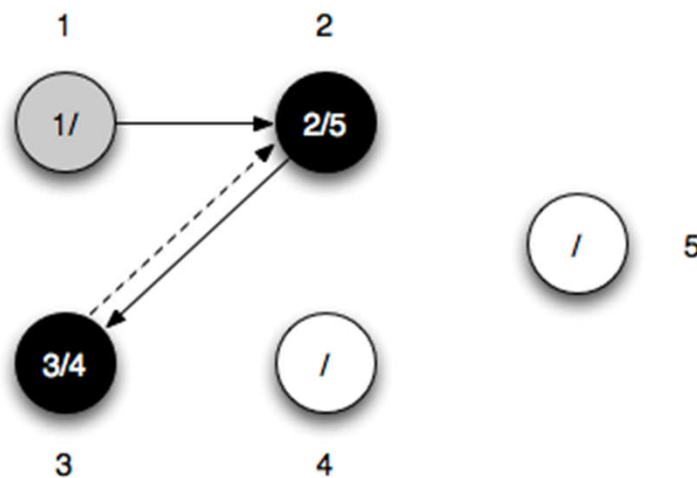
	1	2	3	4	5
d	1	2	3	/	/
f	/	/	/	/	/
π	/	2	3	/	/

Timestep 4: there are no other edges to follow so $u_3.f = 4$



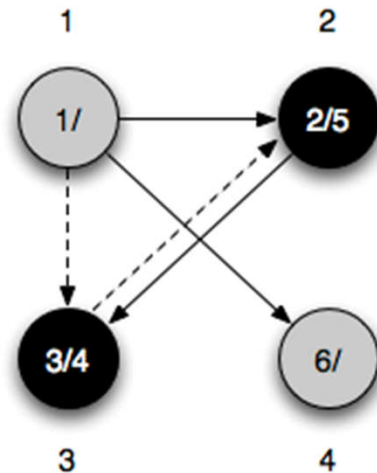
	1	2	3	4	5
d	1	2	3	/	/
f	/	/	4	/	/
π	/	2	3	/	/

Timestep 5: there are no other edges to follow so $u_2.f = 5$



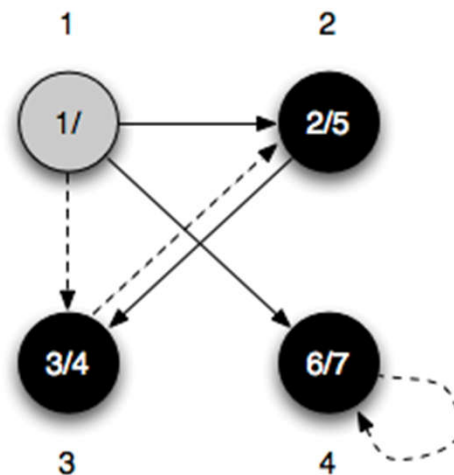
	1	2	3	4	5
d	1	2	3	/	/
f	/	5	4	/	/
π	/	2	3	/	/

Timestep 6: follow edge (u_1, u_4) (note that edge (u_1, u_3) is a *forward edge*) so $u_4.d = 6$



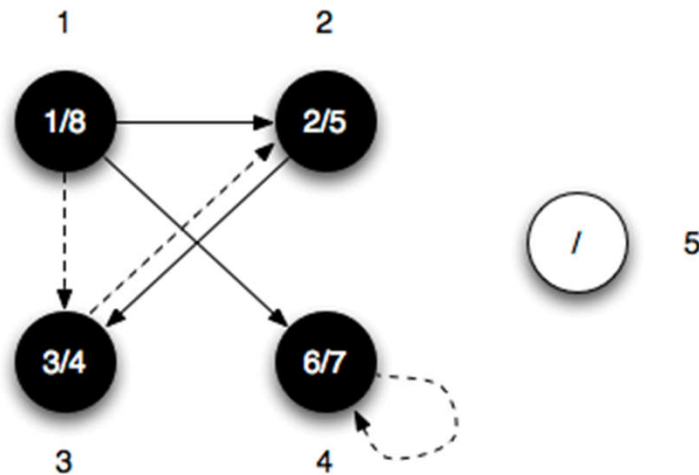
	1	2	3	4	5
d	1	2	3	6	/
f	/	5	4	/	/
π	/	2	3	1	/

Timestep 7: there are no other edges to follow (note that edge (u_4, u_4) is a *back edge*) so $u_4.f = 7$



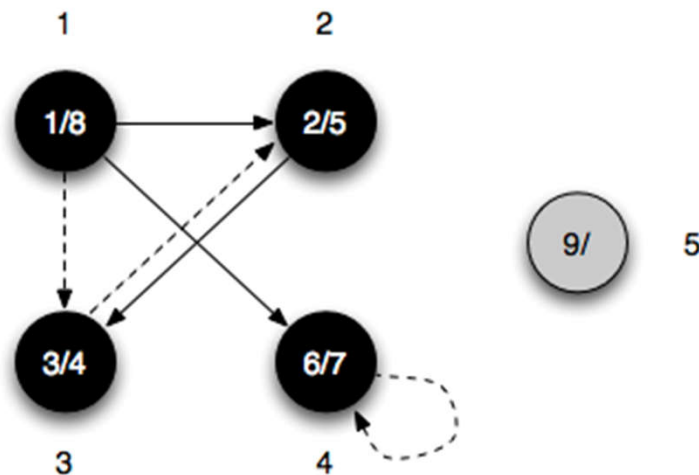
	1	2	3	4	5
d	1	2	3	6	/
f	/	5	4	7	/
π	/	2	3	1	/

Timestep 8: there are no other edges to follow so $u_1.f = 8$



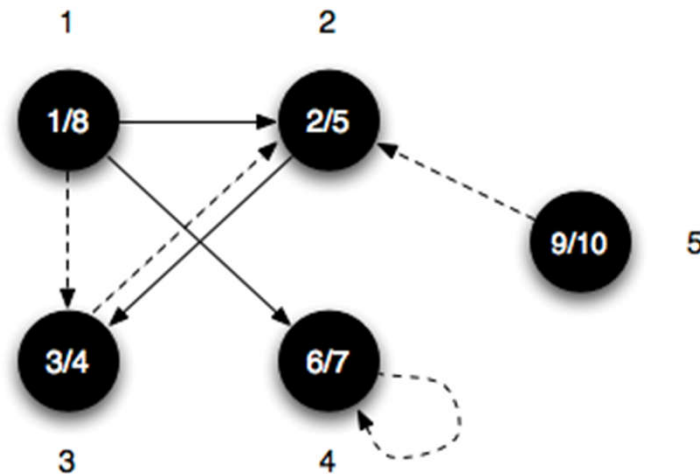
	1	2	3	4	5
d	1	2	3	6	/
f	8	5	4	7	/
π	/	2	3	1	/

Timestep 9: there are no other edges to follow so begin a new tree at $u_5.d = 9$



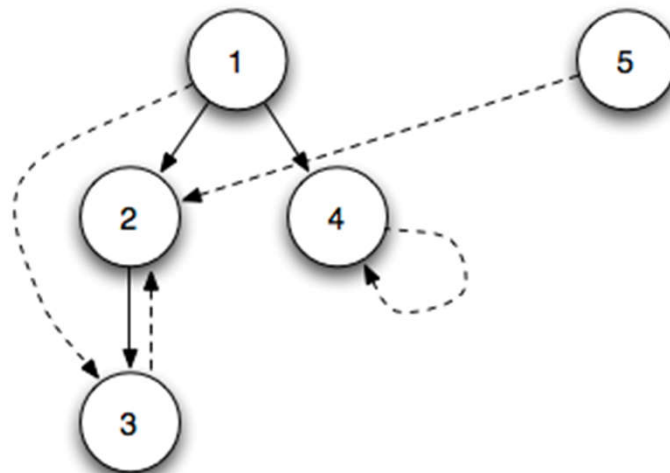
	1	2	3	4	5
d	1	2	3	6	9
f	8	5	4	7	/
π	/	2	3	1	/

Timestep 10: there are no other edges to follow (note that edge (u_5, u_2) is a *cross edge*) so $u_5.f = 10$



	1	2	3	4	5
d	1	2	3	6	9
f	8	5	4	7	10
π	/	2	3	1	/

At this point there are no other **white** vertices so DFS terminates. The final depth-first trees are



Implementation

```
typedef pair<int, int> ii; // In this chapter, we will frequently use these
typedef vector<ii> vii; // three data type shortcuts. They may look cryptic
typedef vector<int> vi;   // but they are useful in competitive programming

vi dfs_num; // global variable, initially all values are set to UNVISITED

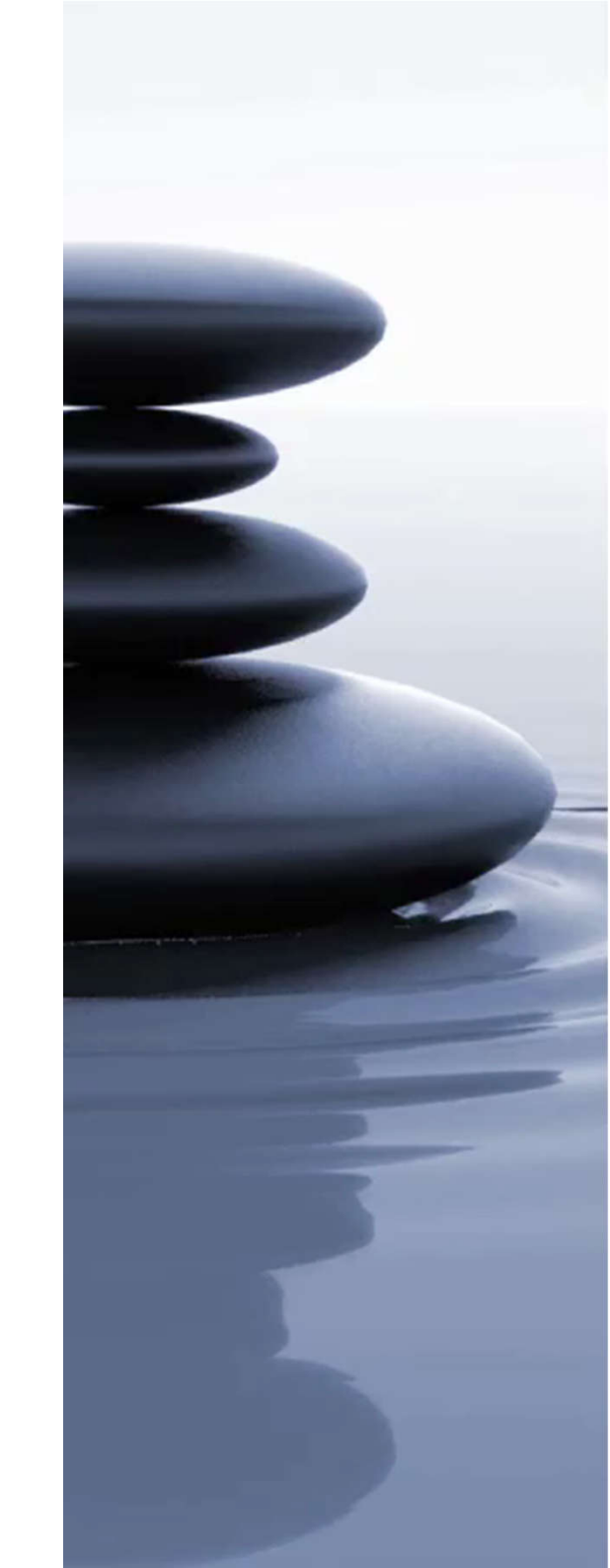
void dfs(int u) { // DFS for normal usage: as graph traversal algorithm
    dfs_num[u] = VISITED; // important: we mark this vertex as visited
    for (int j = 0; j < (int)AdjList[u].size(); j++) { // default DS: AdjList
        ii v = AdjList[u][j]; // v is a (neighbor, weight) pair
        if (dfs_num[v.first] == UNVISITED) // important check to avoid cycle
            dfs(v.first); // recursively visits unvisited neighbors of vertex u
    } // for simple graph traversal, we ignore the weight stored at v.second
```



```
vector<vector<int>> adj; // graph represented as an adjacency list  
int n; // number of vertices
```

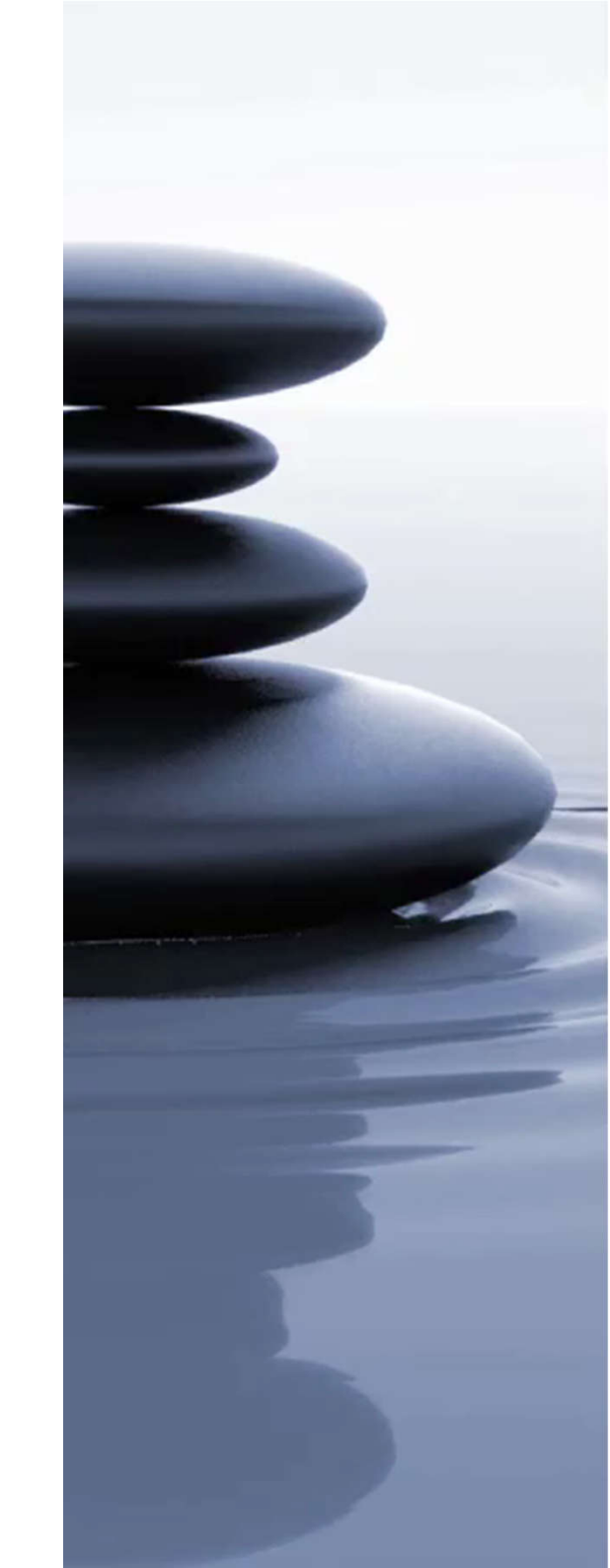
```
vector<bool> visited;
```

```
void dfs(int v) {  
    visited[v] = true;  
    for (int u : adj[v]) {  
        if (!visited[u])  
            dfs(u);  
    }  
}
```



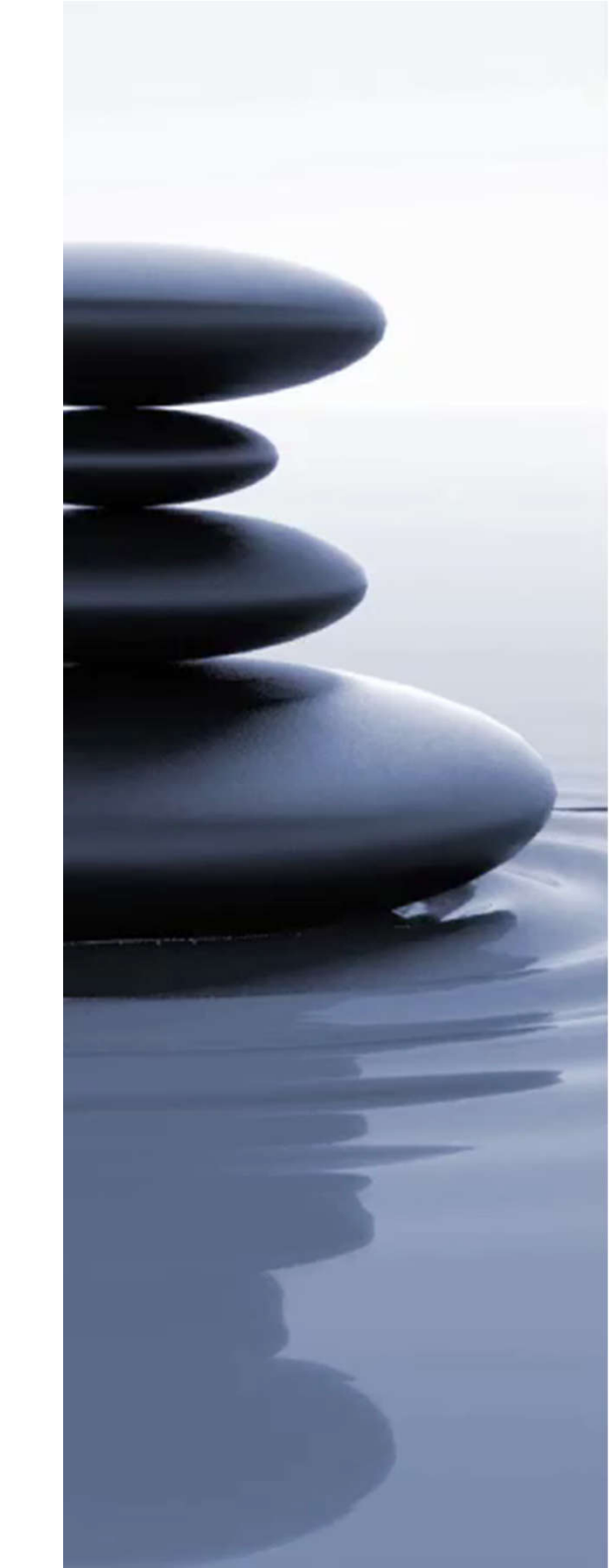
Applications

- Find any path in the graph from source vertex u to all vertices.
- Find lexicographical first path in the graph from source u to all vertices.
- Check if a vertex in a tree is an ancestor of some other vertex:
- At the beginning and end of each search call we remember the entry and exit "time" of each vertex. Now you can find the answer for any pair of vertices (i, j) in $O(1)$: vertex i is an ancestor of vertex j if and only if $\text{entry}[i] < \text{entry}[j]$ and $\text{exit}[i] > \text{exit}[j]$.
- Find the lowest common ancestor (LCA) of two vertices.
- Topological sorting:
- Run a series of depth first searches so as to visit each vertex exactly once in $O(n+m)$ time. The required topological ordering will be the vertices sorted in descending order of exit time.
- Check whether a given graph is acyclic and find cycles in a graph. (As mentioned above by counting back edges in every connected components).
- Find strongly connected components in a directed graph:
- First do a topological sorting of the graph. Then transpose the graph and run another series of depth first searches in the order defined by the topological sort. For each DFS call the component created by it is a strongly connected component.
- Find bridges in an undirected graph:
- First convert the given graph into a directed graph by running a series of depth first searches and making each edge directed as we go through it, in the direction we went. Second, find the strongly connected components in this directed graph. Bridges are the edges whose ends belong to different strongly connected components.



Problems

- [SPOJ: ABCPATH](#)
- [SPOJ: EAGLE1](#)
- [Codeforces: Kefa and Park](#)
- [Timus: Werewolf](#)
- [Timus: Penguin Avia](#)
- [Timus: Two Teams](#)
- [SPOJ - Ada and Island](#)
- [UVA 657 - The die is cast](#)
- [SPOJ - Sheep](#)
- [SPOJ - Path of the Righteous Man](#)
- [SPOJ - Validate the Maze](#)
- [SPOJ - Ghosts having Fun](#)
- [Codeforces - Underground Lab](#)
- [DevSkill - Maze Tester \(archived\)](#)
- [DevSkill - Tourist \(archived\)](#)
- [Codeforces - Anton and Tree](#)
- [Codeforces - Transformation: From A to B](#)
- [Codeforces - One Way Reform](#)
- [Codeforces - Centroids](#)
- [Codeforces - Generate a String](#)
- [Codeforces - Broken Tree](#)
- [Codeforces - Dasha and Puzzle](#)
- [Codeforces - Making genome In Berland](#)
- [Codeforces - Road Improvement](#)
- [Codeforces - Garland](#)
- [Codeforces - Labeling Cities](#)
- [Codeforces - Send the Fool Further!](#)
- [Codeforces - The tag Game](#)
- [Codeforces - Leha and Another game about graphs](#)
- [Codeforces - Shortest path problem](#)
- [Codeforces - Upgrading Tree](#)
- [Codeforces - From Y to Y](#)
- [Codeforces - Chemistry in Berland](#)
- [Codeforces - Wizards Tour](#)
- [Codeforces - Ring Road](#)
- [Codeforces - Mail Stamps](#)
- [Codeforces - Ant on the Tree](#)
- [SPOJ - Cactus](#)
- [SPOJ - Mixing Chemicals](#)

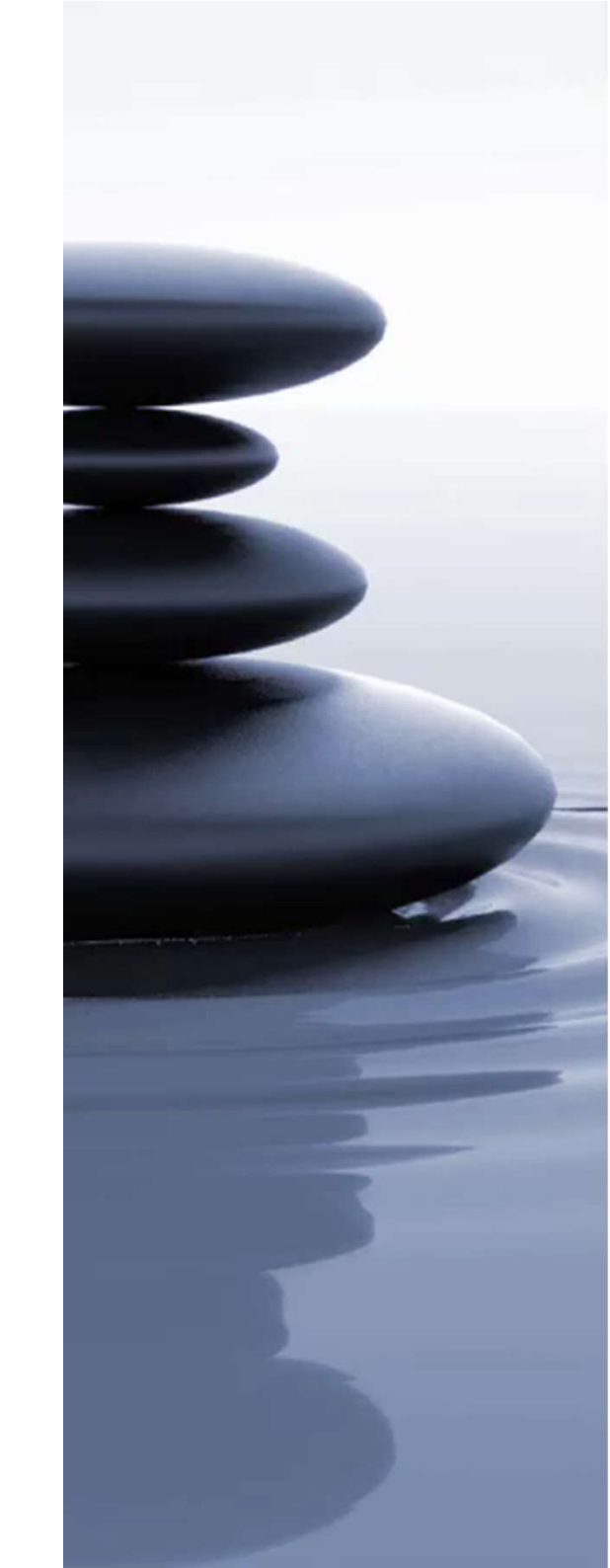


BFS vs DFS

BFS is used to find **the shortest paths** (by number of edges) to every reachable vertex from a given one.

Alternatively we may wish to follow a particular path to maximal depth and then **backtrack** to follow other paths in a depth-first fashion (DFS) creating a forest of depth-first trees.

DFS does not give shortest paths but is useful as part of other algorithms.



Exercises

UVa 10116 - Robot Motion

UVa 11831 - Sticker Collector

UVa 11906 - Knight in a War Grid

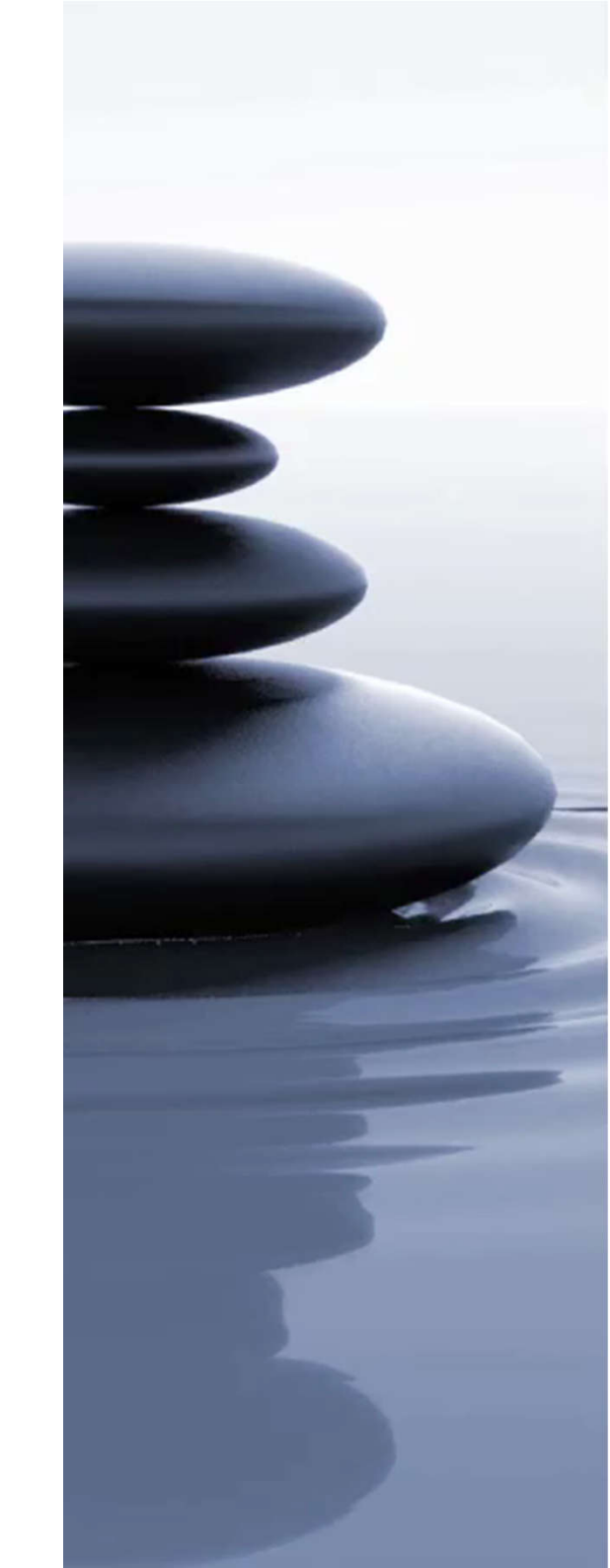
UVa 12442 - Forwarding Emails



**Connected components, bridges,
articulations points**

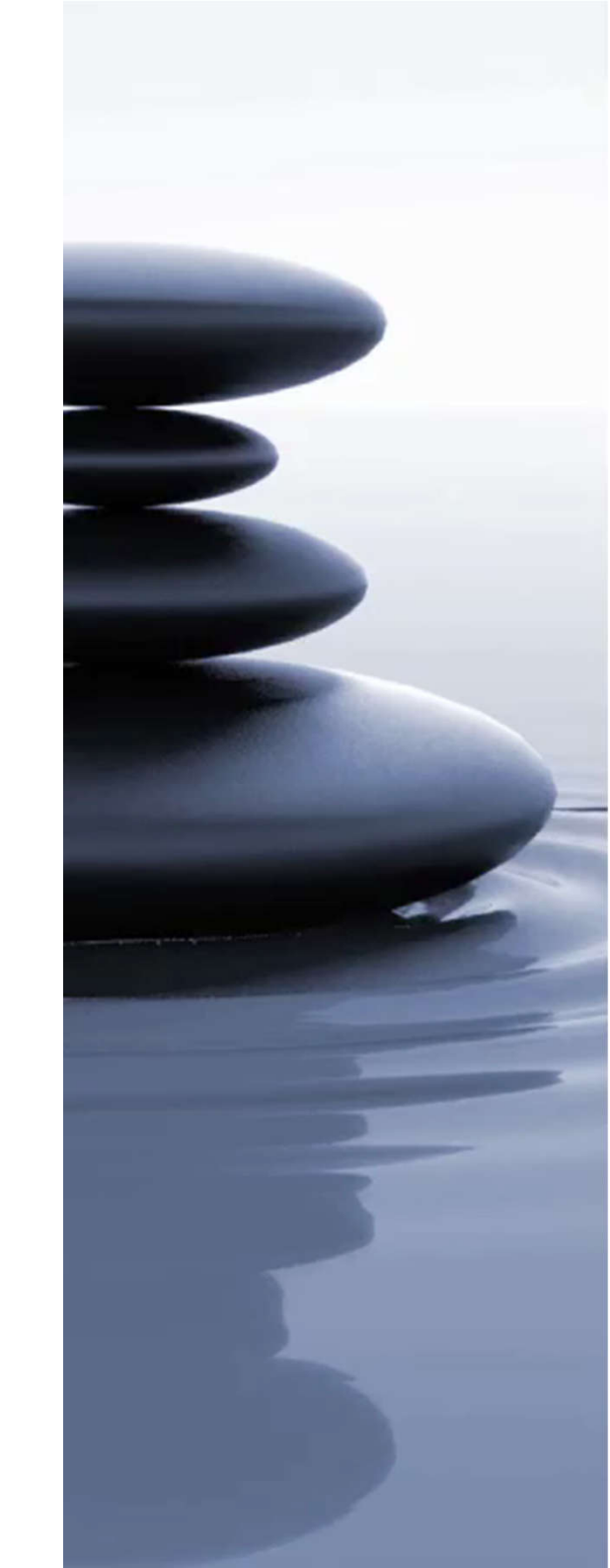


FINDING CONNECTED COMPONENTS



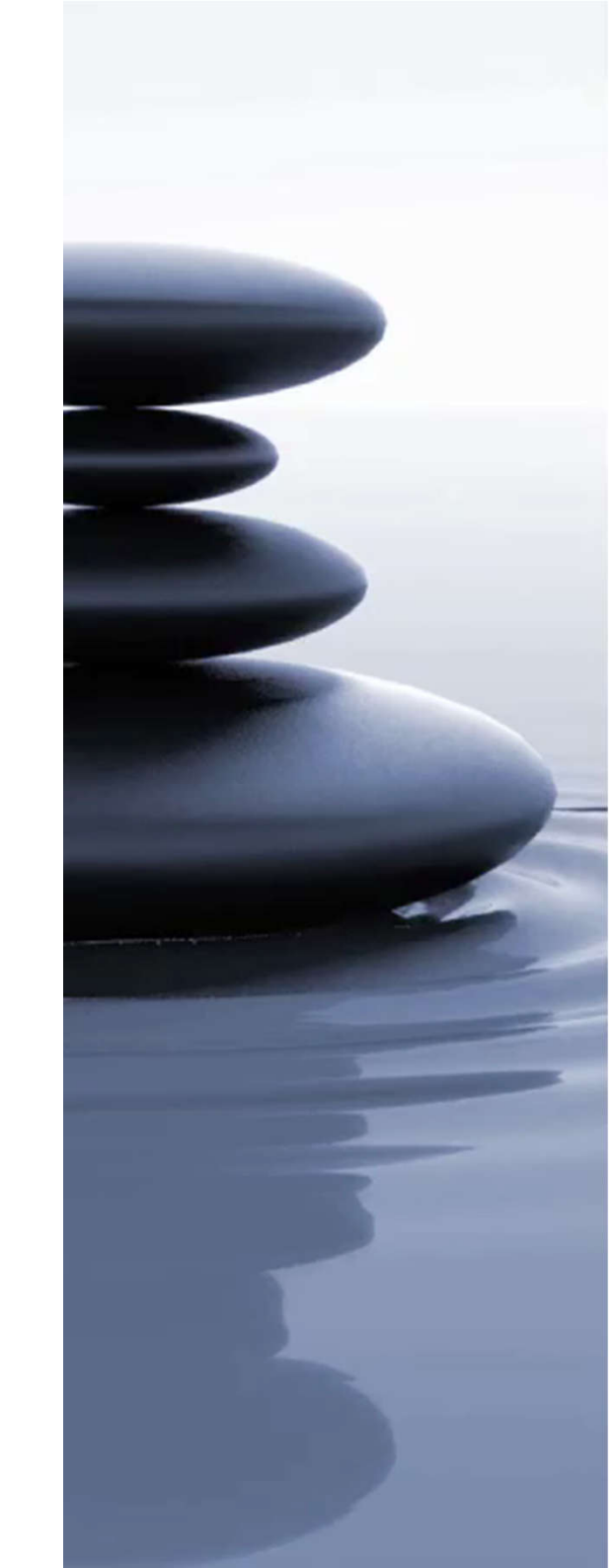
Task

Given an undirected graph with n nodes and m edges. We are required to find in it all the connected components, i.e, several groups of vertices such that within a group each vertex can be reached from another and no path exists between different groups.



Algorithm

- To solve the problem, we can use Depth First Search
- In fact, we will be doing a series of rounds of DFS: The first round will start from first node and all the nodes in the first connected component will be traversed (found). Then we find the first unvisited node of the remaining nodes, and run Depth First Search on it, thus finding a second connected component. And so on, until all the nodes are visited.
- The total asymptotic running time of this algorithm is $O(n+m)$: In fact, this algorithm will not run on the same vertex twice, which means that each edge will be seen exactly two times (at one end and at the other end).

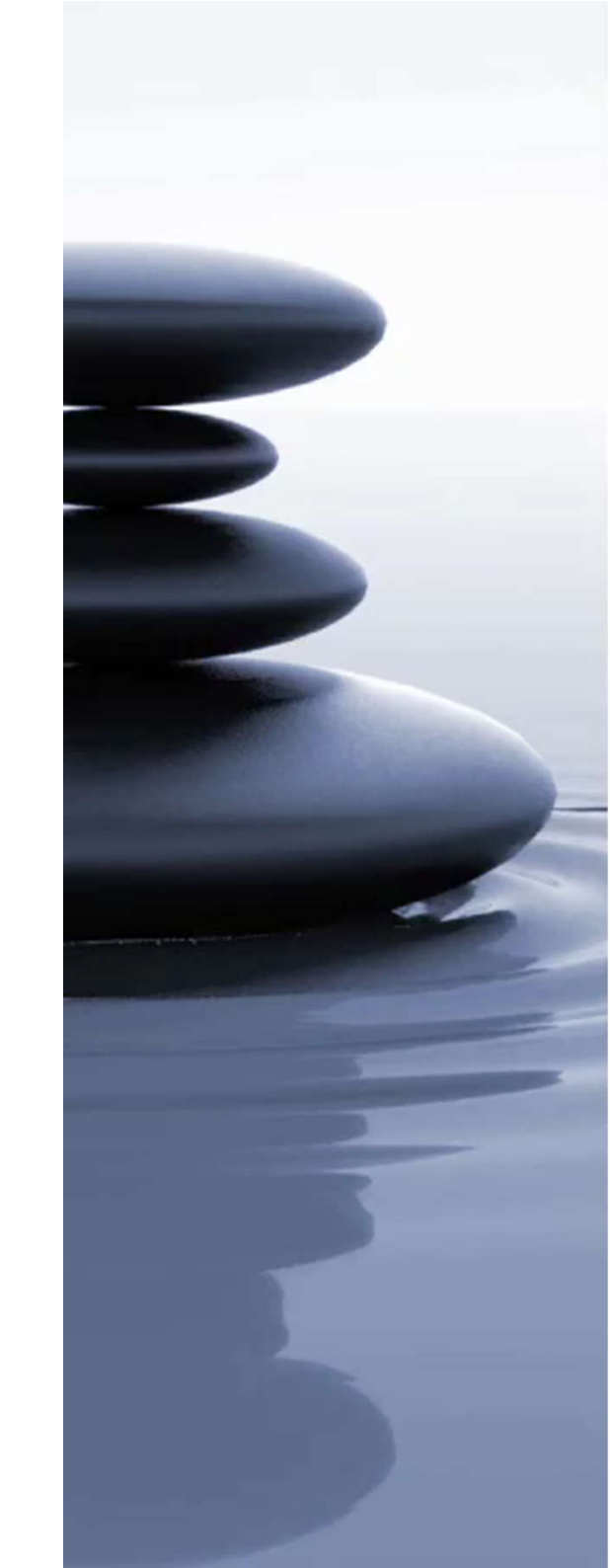


Implementation

```
int n;
vector<int> g[MAXN] ;
bool used[MAXN] ;
vector<int> comp ;

void dfs(int v) {
    used[v] = true ;
    comp.push_back(v);
    for (size_t i = 0; i < (int) g[v].size(); ++i) {
        int to = g[v][i];
        if (!used[to])
            dfs(to);
    }
}

void find_comps() {
    for (int i = 0; i < n ; ++i)
        used [i] = false;
    for (int i = 0; i < n ; ++i)
        if (!used[i]) {
            comp.clear();
            dfs(i);
            cout << "Component:" ;
            for (size_t j = 0; j < comp.size(); ++j)
                cout << ' ' << comp[j];
            cout << endl ;
        }
}
```



Problems

- [SPOJ: CCOMPS](#)
- [SPOJ: CT23E](#)
- [CODECHEF: GERALD07](#)
- [CSES : Building Roads](#)